

2026 전국기능경기대회 채점기준

1. 채점상의 유의사항	직 종 명	클라우드컴퓨팅
※ 다음 사항을 유의하여 채점하시오. 1) AWS의 지역은 ap-northeast-2를 사용합니다. 2) 웹페이지 접근은 크롬이나 파이어폭스를 이용합니다. 3) 웹페이지에서 언어에 따라 문구가 다르게 보일 수 있습니다. 4) shell에서의 명령어의 출력은 버전에 따라 조금 다를 수 있습니다. 5) 문제지와 채점지에 있는 < > 는 변수입니다. 해당 부분을 변경해 입력합니다. 6) 채점은 문항 순서대로 진행해야 합니다. 7) 삭제된 채점자료는 되돌릴 수 없음으로 유의하여 진행하며, 이의신청까지 완료 이후 선수가 생성한 클라우드 리소스를 삭제합니다. 8) 부분 점수가 있는 문항은 채점 항목에 부분 점수가 적혀져 있습니다. 9) 부분 점수가 따로 없는 문항은 모두 맞아야 점수로 인정됩니다. 10) 리소스의 정보를 읽어오는 채점항목은 기본적으로 스크립트 결과를 통해 채점을 진행하며, 만약 선수가 이의가 있다면 명령어를 직접 입력하여 확인해볼 수 있습니다. 11) (예상 출력)은 바로 이전 (명령어 입력)의 예상 출력을 의미합니다. 12) 채점 시에는 별도로 제공한 채점 스크립트(mark.sh)를 실행하여 채점할 수 있습니다. 다만, 선수가 직접 입력을 원할 경우 채점기준표에 명시된 명령어 그대로 입력하여 채점할 수 있습니다. 채점 스크립트는 root 경로에 지정하도록 합니다. 13) 배포된 채점 스크립트(mark.sh) 는 cloudshell-user의 최상위 경로에 위치 하도록 합니다. 14) 모든 채점 사항은 CloudShell에서 진행합니다. 15) 예상 출력의 정답 기준이 '정확히 일치'로 명시되어 있더라도, 값의 일부분이 붉은색으로 표시된 경우에는 해당 붉은색 텍스트만 채점 대상으로 간주하여 진행합니다. 16) 예상 출력 부분에 특정 채점 기준이나 정답 인정 조건이 기재되어 있다면, 반드시 해당 조건을 따라 평가합니다.		

2. 채점기준표

1) 주요항목별 배점				직 종 명		클라우드컴퓨팅		
과제 번호	일련 번호	주요항목	배점	채점방법		채점시기		비고
				독립	합의	경기 진행중	경기 종료후	
제1과제	1	Network Configuration	1	○			○	
	2	Simple Storage Service	2	○			○	
	3	Elastic Container Registry	1	○			○	
	4	NoSQL Database	1	○			○	
	5	Elastic Kubernetes Service	5	○			○	
	6	Lambda Function	1	○			○	
	7	Load Balancing	2.5	○			○	
	8	CloudFront	6.5	○			○	
	9	Application	6	○			○	
	10	Monitoring	1	○			○	
합 계			27					

<monitoring 3문제 제거하여 3점 추가 과제 필요>

2) 채점방법 및 기준

과제 번호	일련 번호	주요항목	일련 번호	세부항목(채점방법)	배점
1과제	1	Network Configuration	1	Resources CIDR	0.5
			2	Routing Tables	0.5
	2	Simple Storage Service	1	S3 Bucket & Objects	1
			2	S3 Configuration	1
	3	Elastic Container Registry	1	ECR Repository & Image	1
	4	NoSQL Database	1	DynamoDB Configuration	1
	5	Elastic Kubernetes Service	1	Cluster Configuration	1
			2	Cluster Encryption & Networking	1
			3	Cluster Node Configuration	1.5
			4	Cluster Pod Configuration	1.5
	6	Lambda Function	1	Function Configuration	1
	7	Load Balancing	1	ALB Configuration	1
			2	ALB Rules Configuration	1.5
	8	CloudFront	1	Distribution Configuration	1
			2	Origin Configuration	1.5
			3	Distribution Policy Configuration	1.5
			4	Custom Headers	1
			5	Static Web Hosting	1.5
	9	Application	1	Application Operation Test - POST	1.5
			2	Application Operation Test - GET	1.5
			3	Application Operation Test - ERROR	1.5
			4	Application Operation Test	1.5
	10	Monitoring	1	Grafana Dashboard Check	1

3) 채점내용

순번	사전준비	
0	1) Cloudshell VPC Environment을 생성 후 접속합니다. - Subnet : wskorea26-priv-subnet-d - Security Group : wskorea26-vpc-environment-sg 2) rm -rf ~/.aws를 진행합니다. 3) aws configure를 입력하고 default.region을 ap-northeast-2로 설정합니다.	
	1) Cloudshell 콘솔에 아래 명령어를 입력합니다. <pre>\$ aws eks update-kubeconfig --region ap-northeast-2 --name wskorea26-cluster 2>/dev/null; CF_ID=\$(aws cloudfront list-distributions --query "DistributionList.Items[?Comment=='wskorea26-concert-cf'].Id [0]" --output text); CF_DOMAIN=\$(aws cloudfront get-distribution --id \$CF_ID --query "Distribution.DomainName" --output text); BUCKET=\$(aws s3api list-buckets --query "Buckets[?starts_with(Name, 'wskorea26-concert-bucket-')].Name [0]" --output text); ALB_ARN=\$(aws elbv2 describe-load-balancers --names wskorea26-book-alb --query "LoadBalancers[0].LoadBalancerArn" --output text); ALB_DNS=\$(aws elbv2 describe-load-balancers --names wskorea26-book-alb --query "LoadBalancers[0].DNSName" --output text); LISTENER_ARN=\$(aws elbv2 describe-listeners --load-balancer-arn \$ALB_ARN --query "Listeners[0].ListenerArn" --output text)</pre>	
순번	채점항목	
1-1	1-1-A (명령어 입력)	<pre>aws ec2 describe-vpcs --filter Name=tag:Name,Values=wskorea26-vpc --query "Vpcs[0].CidrBlock" --output text && aws ec2 describe-subnets --filters "Name=tag:Name,Values=wskorea26-pub-subnet-c,wskorea26-pub-su bnet-d,wskorea26-priv-subnet-c,wskorea26-priv-subnet-d" --query "sort_by(Subnets,&Tags[?Key=='Name'] [0].Value)[].[Tags[?Key=='Na me'] [0].Value,CidrBlock]" --output text</pre>
	1-1-A (예상 출력) 정확히 일치	<pre>172.16.0.0/16 wskorea26-priv-subnet-c 172.16.201.0/24 wskorea26-priv-subnet-d 172.16.202.0/24 wskorea26-pub-subnet-c 172.16.1.0/24 wskorea26-pub-subnet-d 172.16.2.0/24</pre>
1-2	1-2-A (명령어 입력)	<pre>for subnet in wskorea26-pub-subnet-c wskorea26-pub-subnet-d; do aws ec2 describe-route-tables --filters "Name=association.subnet-id,Values=\$(aws ec2 describe-subnets --filters "Name=tag:Name,Values=\$subnet" --query "Subnets[0].SubnetId" --output text)" --query</pre>

		<pre>"RouteTables[0].Tags[?Key=='Name'] [0].Value" --output text; done sort; for subnet in wskorea26-priv-subnet-c wskorea26-priv-subnet-d; do aws ec2 describe-route-tables --filters "Name=association.subnet-id,Values=\$(aws ec2 describe-subnets --filters "Name=tag:Name,Values=\$subnet" --query "Subnets[0].SubnetId" --output text)" --query "RouteTables[0].Tags[?Key=='Name'] [0].Value" --output text; done sort; aws ec2 describe-route-tables --filters "Name=tag:Name,Values=wskorea26-public-rtb" --query "RouteTables[0].Routes[?DestinationCidrBlock=='0.0.0.0/0'].GatewayId [0]" --output text; for rtb in wskorea26-private-rtb-c wskorea26-private-rtb-d; do aws ec2 describe-route-tables --filters "Name=tag:Name,Values=\$rtb" --query "RouteTables[0].Routes[?DestinationCidrBlock=='0.0.0.0/0'].NatGatewa yId [0]" --output text; done</pre>
	1-2-A (예상 출력) 정확히 일치 (순서무관)	<pre>wskorea26-public-rtb wskorea26-public-rtb wskorea26-private-rtb-c wskorea26-private-rtb-d igw-<선수마다 다른 값> nat-<선수마다 다른 값> nat-<선수마다 다른 값, 단 위 nat- 와는 달라야함.></pre>
2-1	2-1-A (명령어 입력)	<pre>echo \$BUCKET && aws s3api list-objects-v2 --bucket "\$BUCKET" --prefix "web/main/" --query "sort(Contents[].Key)" --output text</pre>
	2-1-A (예상 출력) 정확히 일치	<pre>wskorea26-concert-bucket-<선수등번호> web/main/index.html web/main/main.jpeg</pre>
2-2	2-2-A (명령어 입력)	<pre>for key in web/main/index.html web/main/main.jpeg; do kms_arn=\$(aws s3api head-object --bucket "\$BUCKET" --key "\$key" --query "SSEKMSKeyId" --output text); key_id=\$(echo "\$kms_arn" awk -F '/' '{print \$NF}'); aws kms list-aliases --query "Aliases[?TargetKeyId=='\$key_id'].AliasName [0]" --output text; done; aws s3api get-public-access-block --bucket "\$BUCKET" --query "PublicAccessBlockConfiguration.[BlockPublicAcls,IgnorePublicAcls,Bloc kPublicPolicy,RestrictPublicBuckets]" --output text; aws s3api get-bucket-policy-status --bucket "\$BUCKET" --query</pre>

		"PolicyStatus.IsPublic" --output text
	2-2-A (예상 출력) <u>정확히 일치</u>	alias/wskorea26-s3-key alias/wskorea26-s3-key True True True True False
3-1	3-1-A (명령어 입력)	aws ecr describe-repositories --query "repositories[?repositoryName=='wskorea26-book-repo'].repositoryName,imageScanningConfiguration.scanOnPush,encryptionConfiguration.encryptionType]" --output text; aws ecr describe-images --repository-name wskorea26-book-repo --image-ids imageTag=stable --query "imageDetails[0].imageTags" --output text; aws ecr describe-image-scan-findings --repository-name wskorea26-book-repo --image-id imageTag=stable --query "imageScanFindings.findingSeverityCounts" --output json
	3-1-A (예상 출력) <u>정확히 일치</u>	wskorea26-book-repo True KMS stable { "LOW": 1 } <u>Critical 및 High 취약점이 존재하지 않을 경우 정답</u> <u>빈값이 나올 경우 Scan 이력을 확인하여 한번도 진행되지 않은 경우 오답</u> <u>으로 처리합니다.</u>
4-1	4-1-A (명령어 입력)	aws dynamodb describe-table --table-name wskorea26-data-table --query "Table.[TableName,KeySchema[0].[AttributeName,KeyType],DeletionProtectionEnabled]" --output text; aws kms list-aliases --query "Aliases[?TargetKeyId=='\$(aws dynamodb describe-table --table-name wskorea26-data-table --query "Table.SSEDescription.KMSMasterKeyArn" --output text awk -F'/' '{print \$NF}')]'.AliasName [0]" --output text
	4-1-A (예상 출력) <u>정확히 일치</u>	wskorea26-data-table True client_id HASH alias/wskorea26-dynamodb-key
5-1	5-1-A (명령어 입력)	aws eks describe-cluster --name wskorea26-cluster --query "cluster.[name,version]" --output text; aws eks describe-cluster --name wskorea26-cluster --query "sort(cluster.logging.clusterLogging[?enabled==W`trueW`].types[0])"

		--output text
	5-1-A (예상 출력) <u>정확히 일치</u>	wskorea26-cluster 1.35 api audit authenticator controllerManager scheduler
5-2	5-2-A (명령어 입력)	aws kms list-aliases --query "Aliases[?TargetKeyId=='\$(aws eks describe-cluster --name wskorea26-cluster --query 'cluster.encryptedConfig[0].provider.keyArn' --output text awk -F '/' '{print \$NF}')]'.AliasName [0]" --output text; aws ec2 describe-subnets --subnet-ids \$(aws eks describe-cluster --name wskorea26-cluster --query "cluster.resourcesVpcConfig.subnetIds[]" --output text) --query "sort(Subnets[*].Tags[?Key=='Name'].Value[])" --output text
	5-2-A (예상 출력) <u>정확히 일치</u>	alias/wskorea26-eks-key wskorea26-priv-subnet-c wskorea26-priv-subnet-d
5-3	5-3-A (명령어 입력)	for ng in wskorea26-addon-ng wskorea26-app-ng; do aws eks describe-nodegroup --cluster-name wskorea26-cluster --nodegroup-name \$ng --query "nodegroup.[nodegroupName,instanceTypes[0]]" --output textdonefor ng in wskorea26-addon-ng wskorea26-app-ng; do aws ec2 describe-subnets --subnet-ids \$(aws eks describe-nodegroup --cluster-name wskorea26-cluster --nodegroup-name \$ng --query "nodegroup.subnets[]" --output text) --query "sort(Subnets[*].Tags[?Key=='Name'].Value[])" --output textdone
	5-3-A (예상 출력) <u>정확히 일치</u>	wskorea26-addon-ng t3.medium wskorea26-app-ng t3.medium wskorea26-priv-subnet-c wskorea26-priv-subnet-d wskorea26-priv-subnet-c wskorea26-priv-subnet-d
5-4	5-4-A (명령어 입력)	kubectl get namespace wskorea26 --output jsonpath='{.metadata.name}' && echo "" addon_nodes() { kubectl get pods --all-namespaces -o json jq -r ' .items[] select(.metadata.namespace != "wskorea26") select((.metadata.ownerReferences // [] map(.kind) index("DaemonSet")) == null) .spec.nodeName // empty ' }

		<pre> app_nodes() { kubectl get pods -n wskorea26 -o json jq -r ' .items[] select((.metadata.ownerReferences // [] map(.kind) index("DaemonSet")) == null) .spec.nodeName // empty ' }for node in \$(addon_nodes); do kubectl get node "\$node" -o jsonpath='{.metadata.labels.node-type}{"\n"}' done sort -u for node in \$(app_nodes); do kubectl get node "\$node" -o jsonpath='{.metadata.labels.node-type}{"\n"}' done sort -u </pre>
	5-4-A (예상 출력) 정확히 일치	wskorea26 addon app
6-1	6-1-A (명령어 입력)	aws lambda get-function-configuration --function-name wskorea26-book-lambda --query "[FunctionName,Runtime,Environment.Variables.TABLE_NAME]" --output text
	6-1-A (예상 출력) 정확히 일치	wskorea26-book-lambda python3.14 wskorea26-data-table
7-1	7-1-A (명령어 입력)	aws elbv2 describe-load-balancers --names wskorea26-book-alb --query "LoadBalancers[0].[LoadBalancerName,Scheme]" --output text; aws elbv2 describe-listeners --load-balancer-arn \$ALB_ARN --query "Listeners[0].[Port,Protocol]" --output text
	7-1-A (예상 출력) 정확히 일치	wskorea26-book-alb internet-facing 80 HTTP
7-2	7-2-A (명령어 입력)	aws elbv2 describe-rules --listener-arn \$LISTENER_ARN --query "Rules[*].Conditions[*].HttpHeaderConfig.Values[]" --output text; curl -o /dev/null -s -w "%{http_code}\n" http://\$ALB_DNS/book
	7-2-A (예상 출력) 정확히 일치	wskorea26-cf wskorea26-cf 403

		wskorea26-cf는 한번 이상만 출력되면 득점
8-1	8-1-A (명령어 입력)	aws cloudfront get-distribution --id \$CF_ID --query "Distribution.[DomainName,Status]" --output text
	8-1-A (예상 출력) <u>정확히 일치</u>	<Random>.cloudfront.net Deployed
8-2	8-2-A (명령어 입력)	aws cloudfront get-distribution --id \$CF_ID --query "Distribution.DistributionConfig.Origins.Items[].[Id,DomainName]" --output text
	8-2-A (예상 출력) <u>정확히 일치</u>	wskorea26-alb-origin wskorea26-book-alb-<random>.ap-northeast-2.elb.amazonaws.com --- wskorea26-s3-origin wskorea26-concert-bucket-<등번 호>.s3.ap-northeast-2.amazonaws.com
8-3	8-3-A (명령어 입력)	aws cloudfront get-distribution --id \$CF_ID --query "Distribution.DistributionConfig.[DefaultCacheBehavior.TargetOriginId,CacheBehaviors.Items[?PathPattern=='/book*' PathPattern=='/book'].TargetOriginId [0],DefaultCacheBehavior.ViewerProtocolPolicy]" --output text
	8-3-A (예상 출력) <u>정확히 일치</u>	wskorea26-s3-origin wskorea26-alb-origin redirect-to-https
8-4	8-4-A (명령어 입력)	aws cloudfront get-distribution --id \$CF_ID --query "Distribution.DistributionConfig.Origins.Items[.CustomHeaders.Items[.HeaderName,HeaderValue]" --output text
	8-4-A (예상 출력) <u>정확히 일치</u>	X-Origin-Verify wskorea26-cf wskorea26-s3-access true
8-5	8-5-A (명령어 입력)	curl -o /dev/null -s -w "%{http_code}\n" https://\$CF_DOMAIN; curl -o /dev/null -s -w "%{http_code}\n" http://\$CF_DOMAIN/; curl -o /dev/null -s -w "status: %{http_code}, size: %{size_download} bytes\n" https://\$CF_DOMAIN/main.jpeg
	8-5-A (예상 출력) <u>정확히 일치</u>	200 301 status: 200, size: 180926 bytes
9-1	9-1-A (명령어 입력)	curl -s -X POST -H 'Content-Type: application/json' -d '{"client_id":"D1114","username":"akane","email":"akane@ztmy.com","co

		ncert_name":"ZUTOMAYO_INTENSE_II"}' https://\$CF_DOMAIN/book
	9-1-A (예상 출력) <u>정확히 일치</u>	<code>{"booking_id": "77d12a71"}</code> key는 일치 해야하나 value는 다를 수 있음.
9-2	9-2-A (명령어 입력)	<code>curl -s -X GET -H 'Content-Type: application/json'</code> <code>"https://\$CF_DOMAIN/reserv-query?concert_name=ZUTOMAYO_INTENSE_II"</code>
	9-2-A (예상 출력) <u>정확히 일치</u>	<code>[{"username": "akane", "created_at":</code> <code>"2026-06-01T14:53:00.498069+09:00", "email": "akane@ztmy.com",</code> <code>"booking_id": "77d12a71", "client_id": "D1114", "concert_name":</code> <code>"ZUTOMAYO_INTENSE_II"}]</code> booking_id는 9-1에서 나온값과 일치 해야함.
9-3	9-3-A (명령어 입력)	<code>curl -s -o /dev/null -w "%{http_code}\n" -X GET -H</code> <code>'Content-Type: application/json' "https://\$CF_DOMAIN/reserv-query"</code>
	9-3-A (예상 출력) <u>정확히 일치</u>	400
10-1	10-0 (사전 준비)	1) 다음 명령어를 사용해 출력되는 경로로 Grafana 에 접속합니다. <code>aws elbv2 describe-load-balancers --names wskorea26-grafana-alb</code> <code>--query "LoadBalancers[0].DNSName" --output text</code> 2) Grafana에 다음 인증 정보로 로그인을 합니다. <code>userid / password : skills-<등번호>-admin / ₩\$korea26!!</code> 3) wskorea26-monitoring 대시보드에 접근이 가능할 경우. 아래의 10-1~4 채점을 진행합니다
	10-1-A (수동 채점)	대쉬보드에서 book app 파드의 CPU 사용량을 확인할 수 있으면 정답